GeoGuessers

Testing & optimizing spatial calculation algorithms on a laptop

Swayam Goyal Siddharth Choudhary Yakkala Viswanadha Kartikeya

Spatial Informatics | Final project presentation

November 2026

The problem

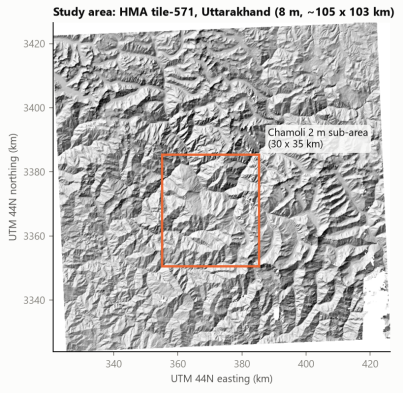
Where do standard terrain tools stop being practical on a laptop, and why?

43 GB
1 m DEM of our area

16 GB
laptop RAM

10.8 B
cells to process

1. **Where is the real bottleneck?**
2. **What does computing on demand from the pyramid cost and save?**
3. **Can it keep up with continuous pan and zoom?**



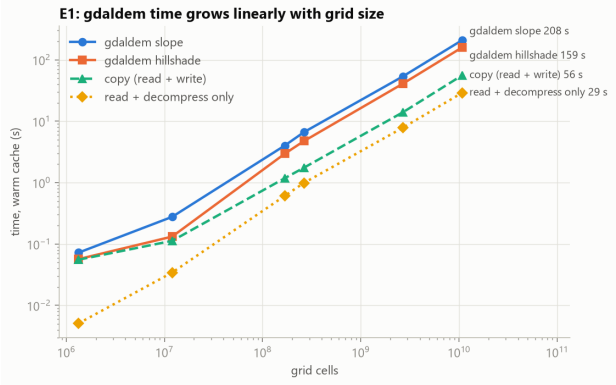
Chamoli, Himalaya: 90 m to 2 m real, 1 m synthetic

What we changed, and why

Each change answers one measured bottleneck

Baseline finding		Our change
Computing runs on one thread	$\rightarrow$	Parallel compiled kernel (Numba)
Decompression and writing block computing	$\rightarrow$	Reader threads + a writer thread
Three products need three passes	$\rightarrow$	One fused kernel
Memory must stay flat	$\rightarrow$	2048^2 windows with a 1-cell halo
Still CPU-limited	$\rightarrow$	GPU engine (CUDA)
99.98% of cells are off-screen	$\rightarrow$	On demand from the pyramid

Baseline: gdaldem doesn't break, it slows down



208 s
slope at 1 m

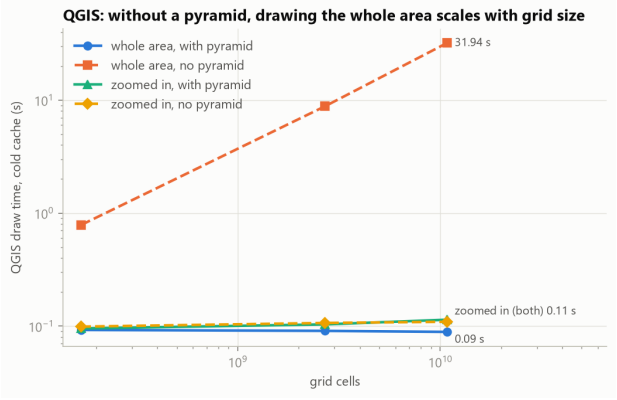
0.87 GB
RAM, at any size

73%
time spent computing

1 / 22
threads used

Not the disk, not memory: one thread computing.

Baseline: for display, the pyramid is everything



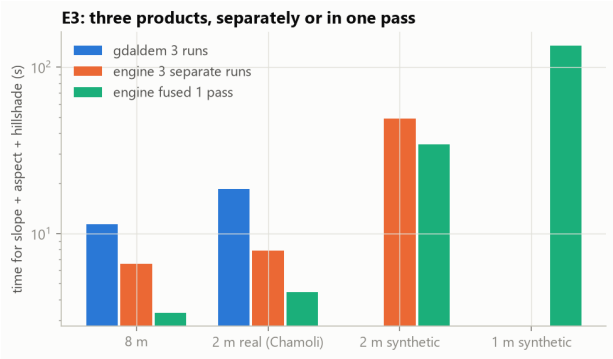
31.9 s
QGIS redraw, no pyramid

0.09 s
with a pyramid

2.9×
smaller with ZSTD (our COG format)

A screen shows ∼2 M pixels.
Why compute 10.8 B cells?

Our CPU engine: parallel, flat memory, exact



3.5×
faster (208 s → 60 s)

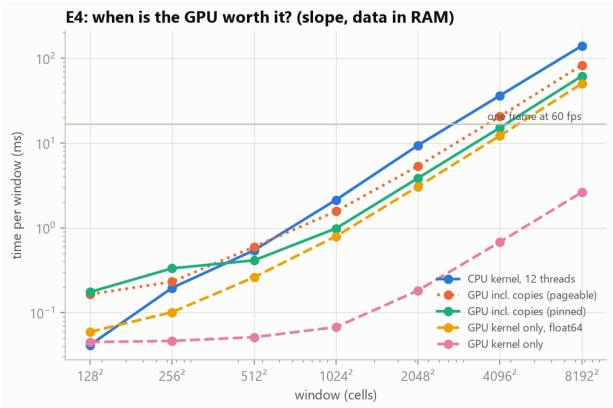
1.4 GB
RAM, at any size

3.4–4.2×
3 products, 1 pass

0
cells differ (gdaldem)

Readers → parallel kernel → writer, in bounded windows.

GPU engine: fast kernel, small overall gain



50×
kernel vs CPU

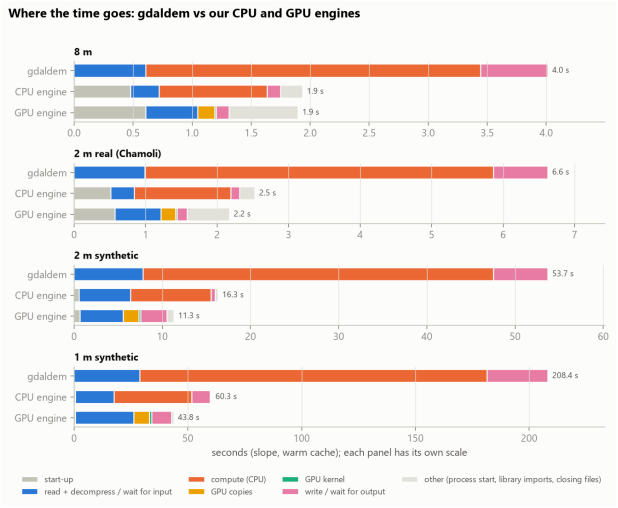
1.4×
whole run vs CPU

95%
of GPU time is copying

4.75×
vs gdaldem (43.8 s)

The GPU computes fast but waits for data.

The bottleneck moves, so compute only what is seen



computation → decompression → writing

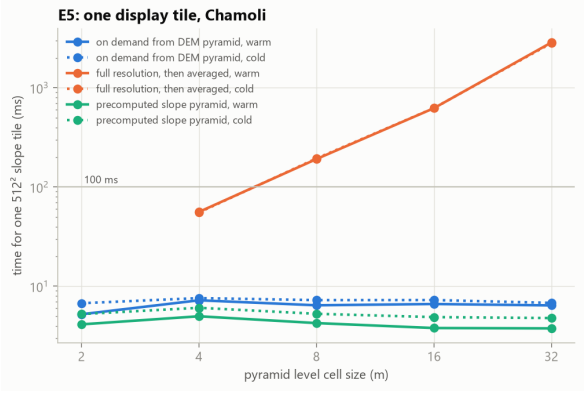
33 s
computing at 1 m

65 s
writing at 1 m

Faster kernels no longer help. Read and write less: derive on demand from the pyramid level that fits the screen.

On-demand derivatives from the pyramid

Compute only what is viewed, at the resolution that fits the screen



Source: E5, final report

5–8 ms
per 512^2 tile, any zoom

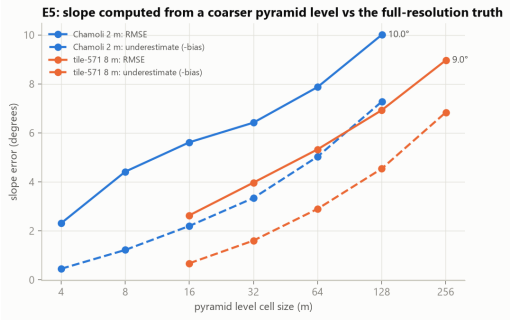
447×
vs exact at 32 m

0 GB
precomputed storage

- Read one 512×512 window (+1 halo) from the COG level that fits the screen
- Exact tiles: 56 ms (4 m) to 2.9 s (32 m)
- New sun angle or product: nothing to rebuild (a slope pyramid costs 1.0 GB per product)

Accuracy vs speed: the pyramid trade-off

The price of speed is slope that is too low; aspect survives well



2.3° RMSE, 1 level up

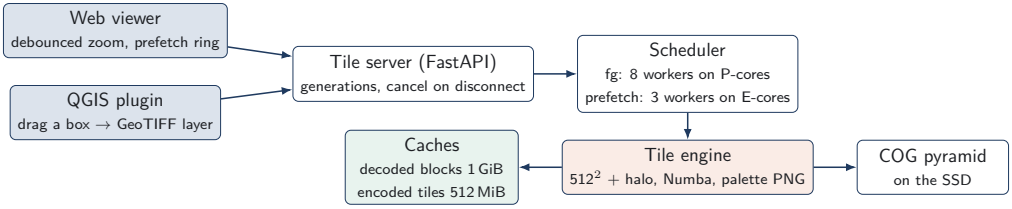
6.4° RMSE, 4 levels up

2–8° aspect median error

- Rule of thumb: $\approx 2.5°$ one level up, $+1.5°$ per further level; almost all **underestimates**
- Steep ground loses most: cells $\geq 45°$ drop from 29% to 22% at 32 m
- A pyramid level beats an independent DEM of the same cell size
- **Auto** mode for display; **Exact** mode for thresholds and hazard maps

Source: E5 against the Chamoli 2 m truth, final report

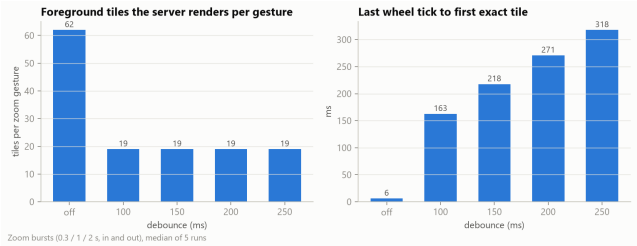
End-to-end system



Block cache	Decoded blocks shared by all tiles; without it 6× more decodes, p95 +40%
Scheduler	Visible tiles first; a visible request promotes a queued prefetch
Cancellation	New generation per pan or zoom; aborted requests dropped in a median 37 ms

Source: `docs/PHASE5_RESULTS.md`, final report

Viewer: the debounce cuts server work 3.3×



62 → 19

server tiles per zoom

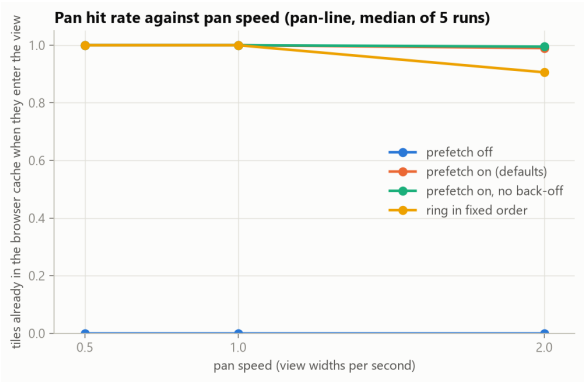
63–71 ms

first exact tile after commit

- Each wheel tick moves the camera at once; cached tiles stand in
- One commit **150 ms** after the last tick
- Notched wheel: 100 ms is as bad as none (36 tiles), 150 ms coalesces (15)

Source: E6, final report

Viewer: prefetch makes panning seamless



98–100%

tiles already there when panned in

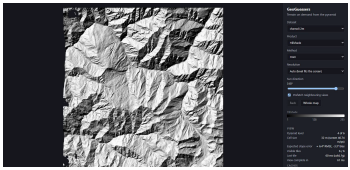
90 vs 95 ms

server p95, prefetch on / off

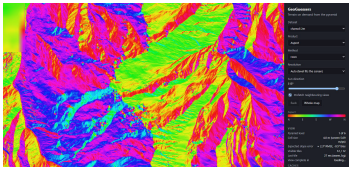
- 8 neighbouring views at low priority, direction of travel first
- Without prefetch: 0% on a straight pan
- **Does not slow visible tiles**: all 122,215 prefetch jobs ran on E-cores
- Server peak 982 MiB; full view 76–158 ms after a zoom

Source: E6, final report

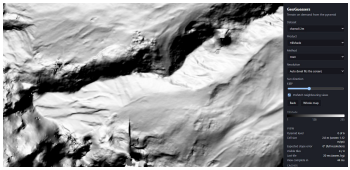
Live demo: the web viewer



Whole map, coarse level



Aspect, on demand



Full resolution (2 m cells)

LIVE `python -m server` then open `http://127.0.0.1:8765/viewer/`

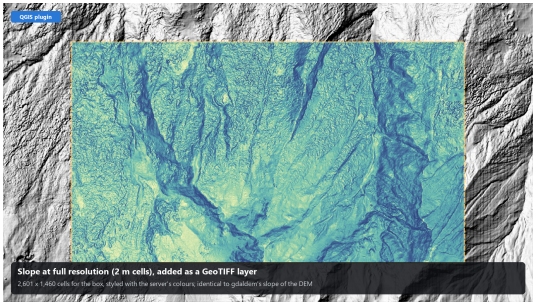
1. Wheel zoom: preview tiles, then one commit
2. Pan: prefetched tiles already there
3. Switch product: slope, aspect, TPI
4. Drag the sun direction (hillshade)
5. Shift + drag a box down to 2 m cells
6. Status panel: level, cell size, expected error

QGIS plugin: native interactive integration

10–225 ms per box (2.85 B-cell DEM)

43 / 44 boxes, 7 products

0 diff vs gdaldem



A 2 m slope box drawn in QGIS, served by the same tile server

- Drag a box → styled GeoTIFF layer, computed in the background
- Level follows the box on screen: 128 m → 2 m as boxes shrink
- Full-resolution box identical to gdaldem (1.22 M cells, max difference 0)
- A 400 M-cell box is refused in 1 ms (limit 64 M cells)

LIVE QGIS → GeoGuessers terrain → Draw a box

Source: `logs/qgis_session/experiment_report.md` (runs 3, 5); final report

Validation: 2.85 billion cells (Swiss Alps lidar)

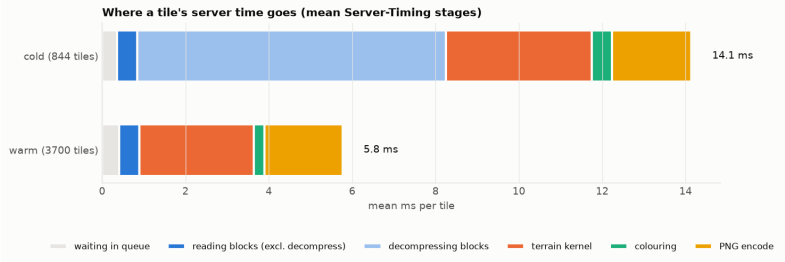
11× the Chamoli dataset, on a second laptop (Ryzen 7 7840HS, Linux, QGIS 4)

6.9–15 ms
median tile, Auto mode

0.01–0.02
server cores (median)

0%
GPU use

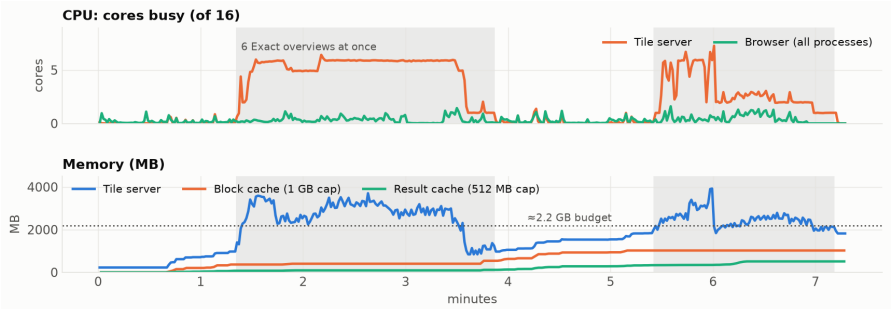
- swissALTI3D 2 m: 60,000 × 47,500 cells, 5.6 GB COG; 7 sessions (viewer, QGIS, both)
- Decompression is 52–55% of a tile read from disk: **decoding, not computing**, is the cost
- CPU time: QGIS 62%, tile server 22%



Where a tile's server time goes (run 1: 844 tiles read from disk, 3,700 with blocks already decoded)

Source: `logs/qgis_session/experiment_report.md`; data © swisstopo

Validation: where it breaks, Exact mode at this scale



Run 4 (7.3 min). Grey: Exact work running. Top: cores busy (tile server, browser); bottom: memory (MB)

- **Auto mode stays at budget:** 1.9–2.3 GB (full caches); ∼3 GB may stay after Exact
- **Exact whole-map overview:** 47–129 s on one core; server memory 2.9–4.9 GB
- **Not cancelled:** each sun-direction change started a new overview; all 8 ran to completion
- **Fix:** build the overview from its own level, or cancel superseded overviews

Source: `logs/qgis_session/experiment_report.md`, findings 1–3

Performance summary

	gdaldem	CPU engine	GPU engine	On demand (viewer)
Slope, 1 m (10.8 B cells)	208 s	60 s	44 s	**5–8 ms per tile**
Speed-up	1×	3.5×	4.75×	**8–450× vs exact tile**
Threads used	1 of 22	12 P-core	CPU + GPU	**0.01–0.02 cores (median)**
Peak memory	0.87 GB	1.4 GB	≤2.0 GiB on GPU	**0.98–2.3 GB**
Full-view redraw	31.9 s (QGIS, no pyramid)	–	–	**76–158 ms after zoom**
Box in QGIS	–	–	–	**10–225 ms** (4.4 s near the 64 M-cell limit)

Engines: whole-grid processing (E1, E3, E4). Viewer: E5 tiles, E6 (Chamoli, peak 982 MiB) and the Swiss runs (2.85 B cells, up to 2.3 GB with full caches; Exact overviews excluded, after which up to ∼3 GB may be kept).

Conclusion

1. **Standard tools slow down, they don't fail.** gdaldem streams 43 GB in 0.87 GB but uses one thread.
2. **Profile before assuming GPU.** Copies are 95% of GPU time; the bottleneck moves from computing to decompressing and writing.
3. **Never compute what cannot be seen.** On demand from the pyramid: 5–8 ms per tile at any zoom, 8–450$\times$ less than exact, 0 GB storage.
4. **Quantify the accuracy cost.** $\approx 2.5^\circ$ slope RMSE one level up, $+1.5^\circ$ per further level; aspect within one compass sector.
5. **Deliver it to the user.** Debounced, prefetching viewer and a QGIS plugin; holds at 2.85 B cells.

Limitations and future work

Limitations

- One study region of steep terrain (larger slope errors than flat land)
- Synthetic 2 m / 1 m grids for timing only
- E6: one client, short scripted traces; Swiss runs hand-driven
- Exact overviews: slow, uncancelled, over the memory budget
- Coarse-level slope bias not corrected

Future work

- Cancel or cache Exact overviews
- Per-level slope bias correction
- Viewshed and terrain distance (G3)
- GPU tile path for bulk exports
- Decompress on the GPU to cut copies (est. up to 1.4×)
- Several clients; cloud-hosted COGs

Code, data manifests, results and reports: `https://github.com/Swayamgoyal/Geoguessers`

Thank you

Questions?

Team GeoGuessers · Codebase and benchmarking harness available